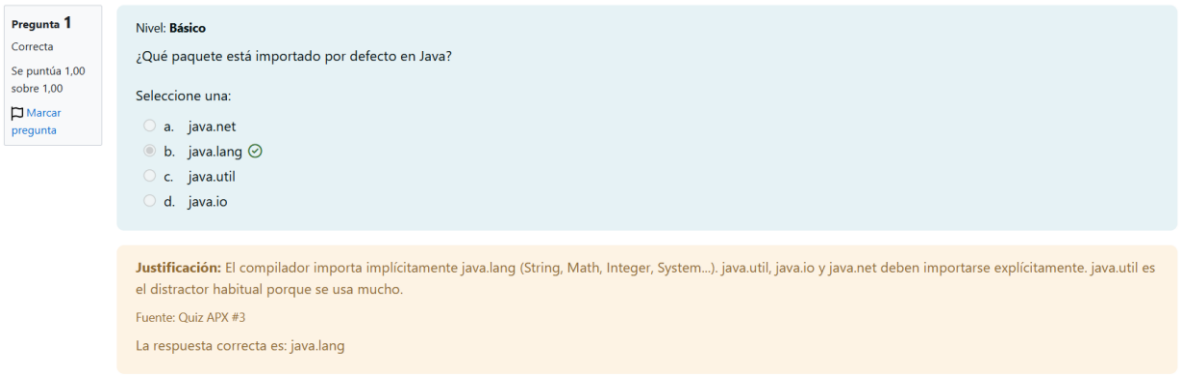
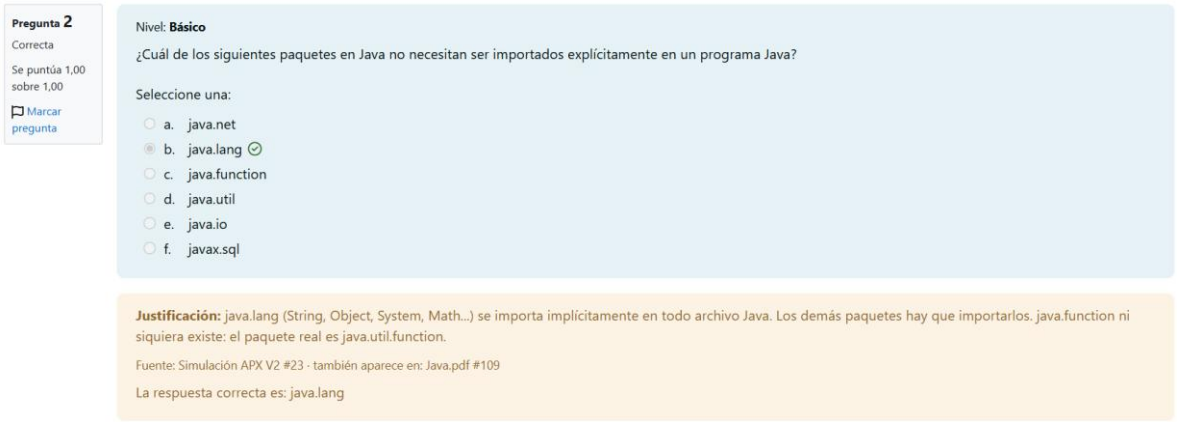
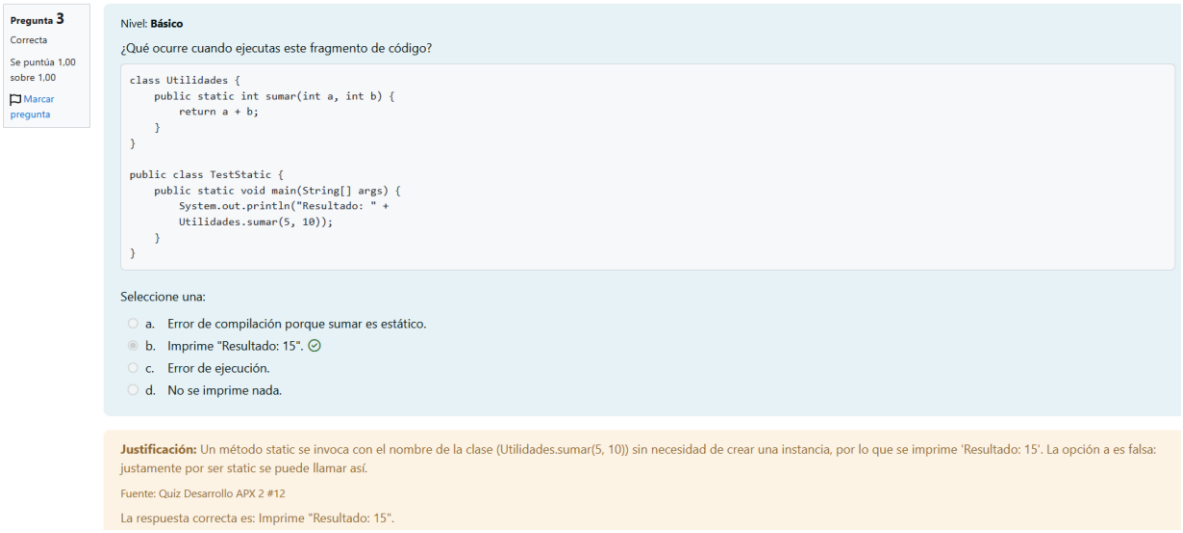


Mi justificación
El paquete java.lang está por default ya que tiene las clases envolventes y el string.



Mi justificación
El paquete java.lang está por default ya que tiene las clases envolventes y el string, entonces no necesita ser importado con import.



Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Qué ocurre cuando ejecutas este fragmento de código?

```
class Utilidades {
    public static int sumar(int a, int b) {
        return a + b;
    }
}

public class TestStatic {
    public static void main(String[] args) {
        System.out.println("Resultado: " +
        Utilidades.sumar(5, 10));
    }
}
```

Seleccione una:

a. Error de compilación porque sumar es estático.

b. Imprime "Resultado: 15". ✓

c. Error de ejecución.

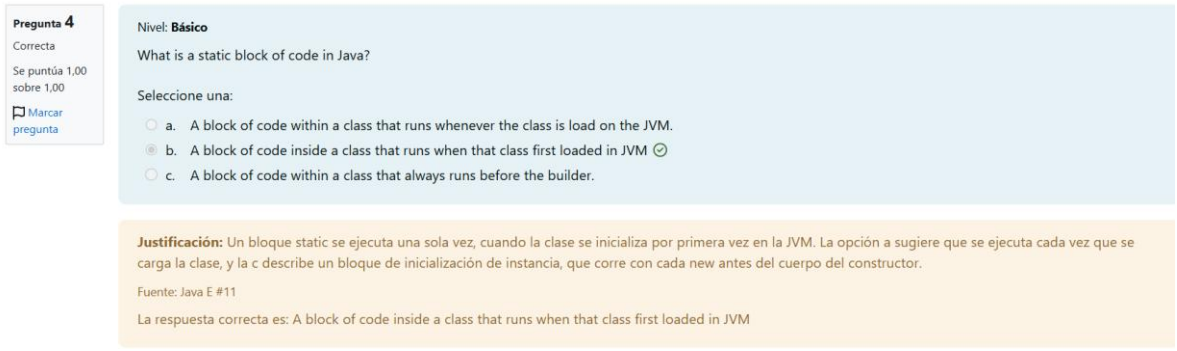
d. No se imprime nada.

Justificación: Un método static se invoca con el nombre de la clase (Utilidades.sumar(5, 10)) sin necesidad de crear una instancia, por lo que se imprime 'Resultado: 15'. La opción a es falsa: justamente por ser static se puede llamar así.

Fuente: Quiz Desarrollo APX 2 #12

La respuesta correcta es: Imprime "Resultado: 15".

Mi justificación
El método sumar es static así que no necesita crearse una instancia para ejecutarse, así que se imprime la suma de 5 y 10 osea 15.



Mi justificación
Los bloques solitos entre llaves {} son la inicialización de la clase, y se corren solo la primera vez que se carga la clase ya sea con instancia o usando un método static dentro de esta.



Mi justificación
La palabra clave protected hace que el método pueda ser usado solo dentro de la misma clase o en una clase del mismo paquete (archivo).

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What two keywords may precede the word 'class' in a class declaration?

Seleccione una o más de una:

- a. Synchronized
- b. Volatile
- c. Static ✓
- d. Local
- e. Public ✓

Justificación: public puede preceder a una clase de nivel superior y static a una clase anidada (static nested class), así que son las dos palabras válidas. synchronized es un modificador de métodos y bloques, y volatile de campos, no de clases. 'local' no es una palabra reservada de Java.

Fuente: Java E #1 · también aparece en: Java DE #21

Las respuestas correctas son: Static, Public

Mi justificación
Las palabras clave syncronized, volatile y local no existen en clases de Java, solo static y public

Pregunta 7

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles son los modificadores que pueden preceder a una declaración de clase de nivel superior?

Seleccione una o más de una:

- a. protected
- b. abstract
- c. strictfp
- d. static
- e. private
- f. public
- g. All the options are correct. ⊗
- h. final

Justificación: Una clase de nivel superior admite public (o ningún modificador, acceso de paquete), abstract, final y strictfp. protected, private y static solo se aplican a miembros o a clases anidadas, así que 'todas' también es falsa.

Fuente: Cuestionario APX V3 #23 · también aparece en: Java.pdf #212

Las respuestas correctas son: abstract, strictfp, public, final

Mi justificación
La verdad no le entendí a la pregunta.

Pregunta 8

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What changes will make this code compile?

```
class X {
    X() { }
    private void one() { }
}
public class Y extends X {
    Y() { }
    private void two() {
        one();
    }
    public static void main(String[] args) {
        new Y().two();
    }
}
```

Seleccione una:

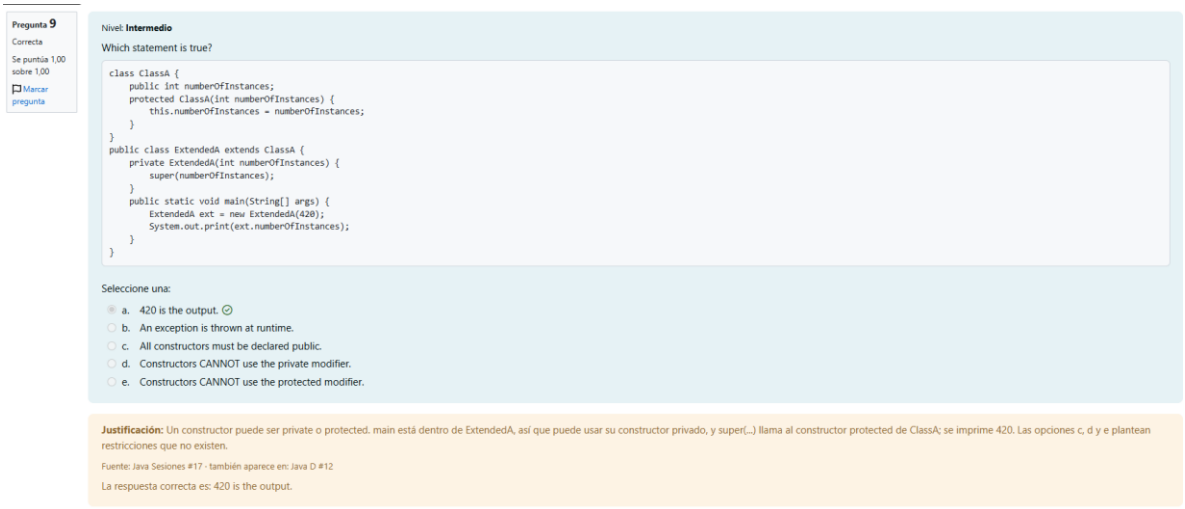
- a. Adding the public modifier to the declaration of class X.
- b. Adding the protected modifier to the X() constructor.
- c. Changing the private modifier on the declaration of the one() method to protected.
- d. Removing the Y() constructor.
- e. Removing the private modifier from the two() method.

Justificación: one() es private en X, así que no se hereda ni es visible desde Y (javac: cannot find symbol one()). Cambiarlo a protected lo hace accesible desde la subclase. Hacer pública la clase X (probado con X en su propio archivo), tocar el constructor de X, quitar Y() o cambiar el modificador de two() no cambia la visibilidad de one(), y el error persiste.

Fuente: Java Sesiones #24 - también aparece en: Java D #11, Java Sesiones #41

La respuesta correcta es: Changing the private modifier on the declaration of the one() method to protected.

Mi justificación
Si le quitas el private a two(), lo vuelves públcio y accesible desde el main.



Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which statement is true?

```
class ClassA {
    public int numberOfInstances;
    protected ClassA(int numberOfInstances) {
        this.numberOfInstances = numberOfInstances;
    }
}
public class ExtendedA extends ClassA {
    private ExtendedA(int numberOfInstances) {
        super(numberOfInstances);
    }
    public static void main(String[] args) {
        ExtendedA ext = new ExtendedA(420);
        System.out.print(ext.numberOfInstances);
    }
}
```

Seleccione una:

a. 420 is the output.

b. An exception is thrown at runtime.

c. All constructors must be declared public.

d. Constructors CANNOT use the private modifier.

e. Constructors CANNOT use the protected modifier.

Justificación: Un constructor puede ser private o protected. main está dentro de ExtendedA, así que puede usar su constructor privado, y super(...) llama al constructor protected de ClassA; se imprime 420. Las opciones c, d y e plantean restricciones que no existen.

Fuente: Java Sesiones #17 · también aparece en: Java D #12

La respuesta correcta es: 420 is the output.

Mi justificación
La instrucción new ExtendedA(420) ejecuta el constructor de Extended (el private), el cual llama al constructor de ClassA (el protected, como es protected sí se puede acceder a él), el cual hace que this.numberOfInstances = numberOfInstances, así que el 420 termina siendo atributo de nuestra instancia ext, al imprimir ext.numberOfInstances obtenemos el mismo 420.



Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Prioridad {
    {
        System.out.println("default");
    }
    static String saludo = "Variable";
    public Prioridad() {
        System.out.println("Constructor");
    }
    static {
        System.out.println("static");
    }
    public void getSaludo() {
        System.out.println(saludo);
    }
    public static void main(String[] args) {
        Prioridad p = new Prioridad();
        p.getSaludo();
    }
}
```

Seleccione una:

a. Constructor Variable

b. static default Constructor Variable ✓

c. static constructor default Variable

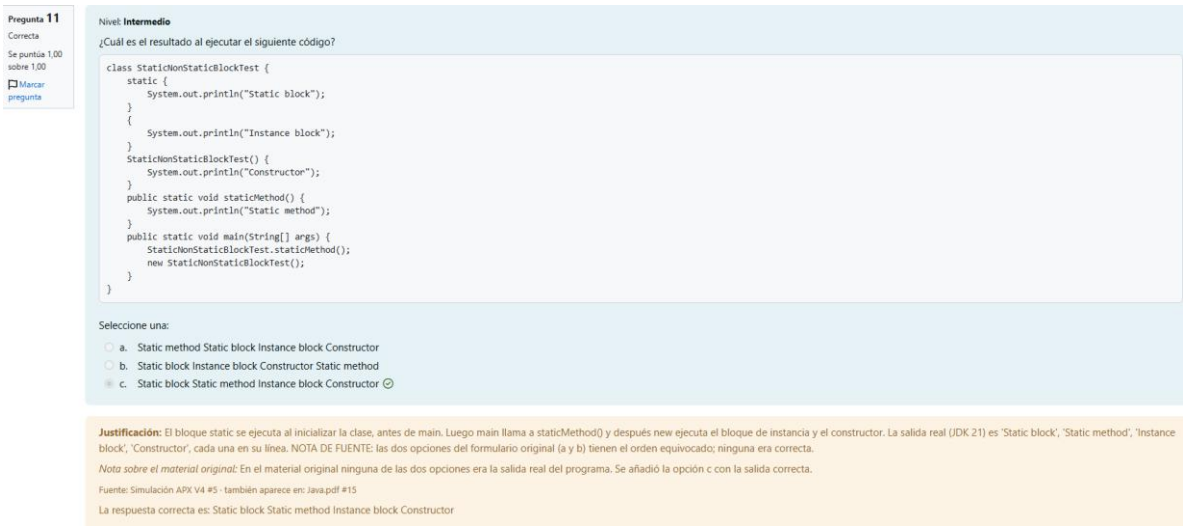
d. Constructor static Variable

Justificación: Al inicializar la clase se ejecutan los inicializadores static en orden (se asigna saludo y se imprime 'static'). Al crear el objeto se ejecuta primero el bloque de instancia ('default') y después el constructor ('Constructor'). Por último getSaludo() imprime 'Variable'. El distractor c invierte el bloque de instancia y el constructor.

Fuente: Quiz Desarrollo APX 2 #20

La respuesta correcta es: static default Constructor Variable

Mi justificación
Yo no sabía que existía el inicializador estático, pero sabía que default al ser el inicializador iría primero, luego el constructor y finalmente la variable que se obtiene con getSaludo(). Por descarte quedaba la b, aunque no sabía que static iría primero.



Pregunta 11
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Intermedio**

¿Cuál es el resultado al ejecutar el siguiente código?

```
class StaticNonStaticBlockTest {
    static {
        System.out.println("Static block");
    }
    {
        System.out.println("Instance block");
    }
    StaticNonStaticBlockTest() {
        System.out.println("Constructor");
    }
    public static void staticMethod() {
        System.out.println("Static method");
    }
    public static void main(String[] args) {
        StaticNonStaticBlockTest.staticMethod();
        new StaticNonStaticBlockTest();
    }
}
```

Seleccione una:

a. Static method Static block Instance block Constructor
b. Static block Instance block Constructor Static method
c. Static block Static method Instance block Constructor

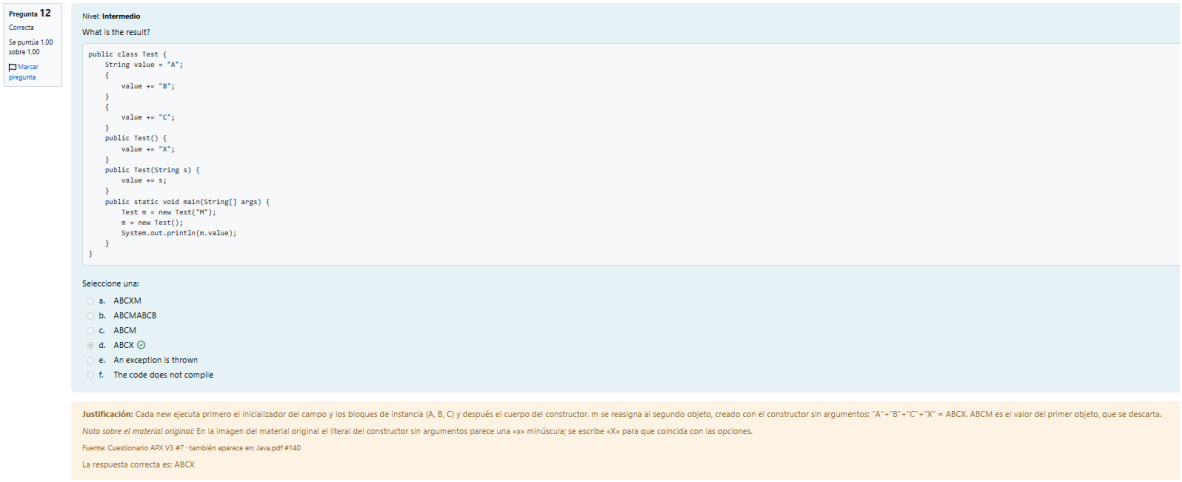
Justificación: El bloque static se ejecuta al inicializar la clase, antes de main. Luego main llama a staticMethod() y después new ejecuta el bloque de instancia y el constructor. La salida real (JDK 21) es 'Static block', 'Static method', 'Instance block', 'Constructor', cada una en su línea. NOTA DE FUENTE: las dos opciones del formulario original (a y b) tienen el orden equivocado; ninguna era correcta.

Nota sobre el material original: En el material original ninguna de las dos opciones era la salida real del programa. Se añadió la opción c con la salida correcta.

Fuente: Simulación APX V4 #5 · también aparece en: Java.pdf #15

La respuesta correcta es: Static block Static method Instance block Constructor

Mi justificación
Yo no sabía que existía el inicializador estático, primero se ejecutaría el método estático al ser llamado por static.Method(), luego el bloque de instancia y finalmente el constructor. Por descarte la c) es la correcta aunque no sabía por qué static block iría primero.



Pregunta 12
Correcta
Se puntúa 1.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class Test {
    String value = "A";
    {
        value += "B";
    }
    {
        value += "C";
    }
    public Test() {
        value += "X";
    }
    public Test(String s) {
        value += s;
    }
    public static void main(String[] args) {
        Test m = new Test("M");
        m = new Test();
        System.out.println(m.value);
    }
}
```

Seleccione una:

a. ABCXM
b. ABCMABCB
c. ABCM
d. ABCX ✓
e. An exception is thrown
f. The code does not compile

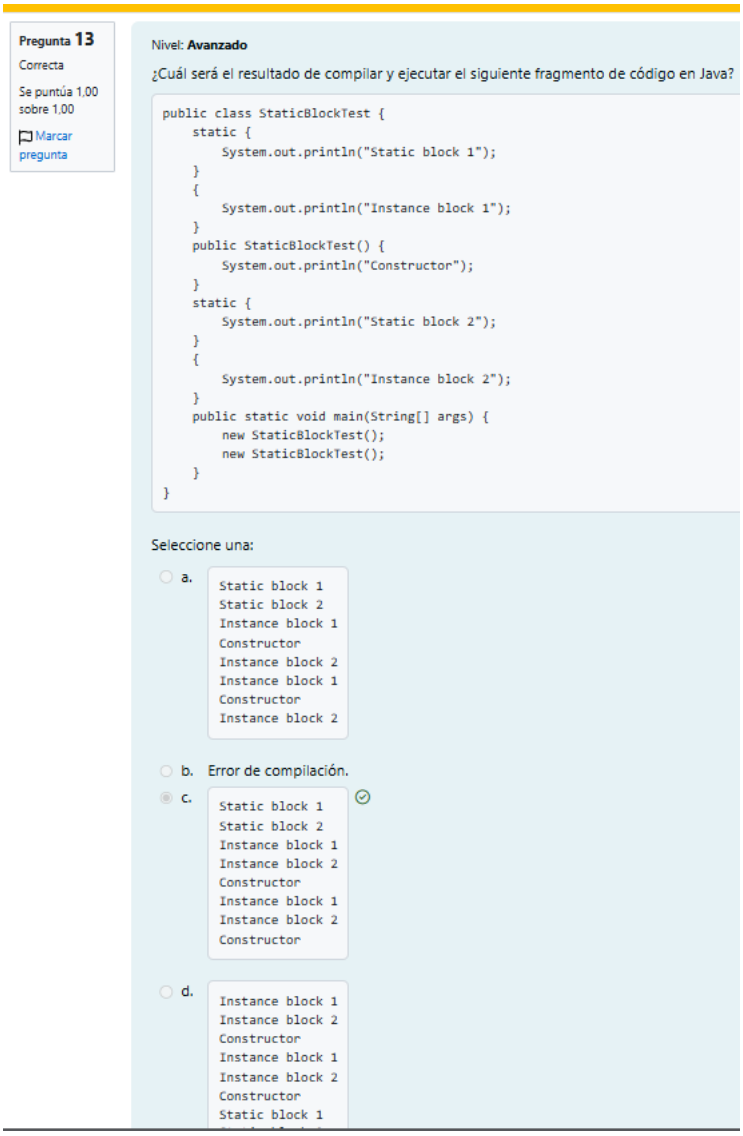
Justificación: Cada new ejecuta primero el inicializador del campo y los bloques de instancia (A, B, C) y después el cuerpo del constructor. m se reasigna al segundo objeto, creado con el constructor sin argumentos: "A"+"B"+"C"+"X" = ABCX. ABCM es el valor del primer objeto, que se descarta.

Nota sobre el material original: En la imagen del material original el literal del constructor sin argumentos parece una «x» minúscula; se escribe «X» para que coincida con las opciones.

Fuente: Cuestionario APX V3 #7 · también aparece en: Java.pdf #140

La respuesta correcta es: ABCX

Mi justificación
El orden es bastante directo. Se define value = “A” en los iniciaizadores se le concatena “B” y luego “C”, y el método de instancia Test() le agrega “X”. Todo esto ocurre en la línea de m = newTest(), la línea anterior queda obsoleta porque se creo un nuevo Test. Así imprimiendo m.value tenemos ABCX.



Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Avanzado**

¿Cuál será el resultado de compilar y ejecutar el siguiente fragmento de código en Java?

```
public class StaticBlockTest {
    static {
        System.out.println("Static block 1");
    }
    {
        System.out.println("Instance block 1");
    }
    public StaticBlockTest() {
        System.out.println("Constructor");
    }
    static {
        System.out.println("Static block 2");
    }
    {
        System.out.println("Instance block 2");
    }
    public static void main(String[] args) {
        new StaticBlockTest();
        new StaticBlockTest();
    }
}
```

Seleccione una:

a.
```
Static block 1
Static block 2
Instance block 1
Constructor
Instance block 2
Instance block 1
Constructor
Instance block 2
```

b. Error de compilación.

c.
```
Static block 1
Static block 2
Instance block 1
Instance block 2
Constructor
Instance block 1
Instance block 2
Constructor
```

d.
```
Instance block 1
Instance block 2
Constructor
Instance block 1
Instance block 2
Constructor
Static block 1
```

Mi justificación
En orden primero van bloques estáticos de inicialización (solo una vez), bloques de instancia, bloques de construcción. Entonces, la creación de dos instancias imprimirían static 1, static 2, instance 1, instance 2, constructor, instance 1, instance 2, constructor.

RESULTADO = 11/13 = 8.5